Chapter 14. The Java Module System

This chapter covers

- The evolutionary forces causing Java to adopt a module system
- The main structure: module declarations and requires and exports directives
- Automatic modules for legacy Java Archives (JARs)
- Modularization and the JDK library
- Modules and Maven builds
- A brief summary of module directives beyond simple `requires` and `exports`

The main and most-discussed new feature introduced with Java 9 is its module system. This feature was developed within project Jigsaw, and its development took almost a decade. This timeline is a good measure of both the importance of this addition and the difficulties that the Java development team met while implementing it. This chapter provides background on why you should care as a developer what a module system is, as well as an overview of what the new Java Module System is intended for and how you can benefit from it.

Note that the Java Module System is a complex topic that merits a whole book. We recommend *The Java Module System* by Nicolai Parlog (Manning Publications, **https://www.manning.com/books/the-java-module-system**) for a comprehensive resource. In this chapter, we deliberately keep to the broad-brush picture so that you understand the main motivation and how to work with Java modules.

asoning about software

Before you delve into the details of the Java Module System, it's useful to understand some motivation and background to appreciate the goals set out by the Java language designers. What does modularity mean? What problem is the module system looking to address? This book has spent

quite a lot of time discussing new language features that help us write code that reads closer to the problem statement and, as a result, is easier to understand and maintain. This concern is a low-level one, however. Ultimately, at a high level (software-architectural level), you want to work with a software project that's easy to reason about because this makes you more productive when you introduce changes in your code base. In the following sections, we highlight two design principles that help produce software that's easier to reason about: *separation of concerns* and *information hiding*.

14.1.1. Separation of concerns

Separation of concerns (SoC) is a principle that promotes decomposing a computer program into distinct features. Suppose that you need to develop an accounting application that parses expenses in different formats, analyzes them, and provides summary reports to your customer. By applying SoC, you split parsing, analysis, and reporting into separate parts called *modules*—cohesive groups of code that have little overlap. In other words, a module groups classes, allowing you to express visibility relationships between classes in your application.

You might say, "Ah, but Java packages already group classes." You're right, but Java 9 modules give you finer-grained control of which classes can see which other classes and allow this control to be checked at compile time. In essence, Java packages don't support modularity.

The SoC principle is useful at an architectural point of view (such as model versus view versus controller) and in a low-level approach (such as separating the business logic from the recovery mechanism). The benefits are

- Allowing work on individual parts in isolation, which helps team collaboration
- Facilitating reuse of separate parts
- Easier maintenance of the overall system

14.1.2. Information hiding

Information hiding is a principle that encourages hiding implementation

portant? In the context of building soft-

e frequently. By hiding implementation de-

es that a local change will require cascad-

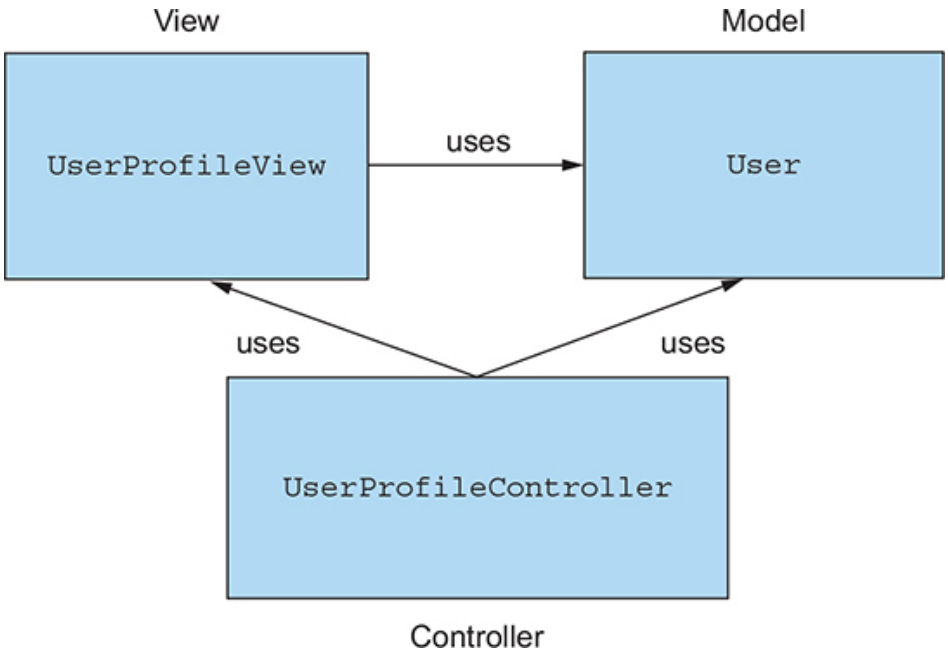
our program. In other words, it's a useful principle for managing and protecting your code. You often hear the term *encapsulation* used to indicate that a specific piece of code is so well isolated from the other parts of the application that changing its internal implementation won't negatively affect them. In Java, you can get a compil-

er to check that *components within a class* are well encapsulated by using the `private` keyword appropriately. But until Java 9, there was no language structure to allow the compiler to check that *classes and packages were available only* for the intended purposes.

14.1.3. Java software

These two principles are fundamental in any well-designed software. How do they fit with Java language features? Java is an object-oriented language, and you work with classes and interfaces. You make your code *modular* by grouping packages, classes, and interfaces that address a specific concern. In practice, reasoning about raw code is a bit abstract. As a result, tools such as UML diagrams (or, more simply, boxes and arrows) help you reason about your software by visually representing dependencies among parts of your code. **Figure 14.1** shows a UML diagram for an application managing a user profile that has been decomposed into three specific concerns.

Figure 14.1. Three separate concerns with dependencies



What about information hiding? In Java, you're familiar with using visibility modifiers to control access to methods, fields, and classes: public, protected, package-level, and private. As we clarify in the next section, however, their granularity isn't fine enough in many cases, and you could be obliged to declare a method public even if you didn't intend to make it oncern wasn't a huge one in the early days dependency chains were relatively small. ns are large, the problem is more important. Indeed, if you see a `public` field or method in a class, you probably feel entitled to use it (don't you?), even though the designer may regard it as being only for private use among a few of his own classes!

Now that you understand the benefits of modularization, you may wonder how supporting it causes changes in Java. We explain in the next section.

14.2. Why the Java Module System was designed

In this section, you learn why a new module system was designed for the Java language and compiler. First, we cover the limitations of modularity before Java 9. Next, we provide background about the JDK library and explain why modularizing it was important.

14.2.1. Modularity limitations

Unfortunately, the built-in support in Java to help produce modular software projects was somewhat limited before Java 9. Java has had three levels at which code was grouped: classes, packages, and JARs. For classes, Java has always had support for access modifiers and encapsulation. There was little encapsulation at the package and JAR levels, however.

Limited Visibility Control

As discussed in the previous section, Java provides access modifiers to support information hiding. These modifiers are public, protected, package-level, and private visibility. But what about controlling visibility *between* packages? Most applications have several packages defined to group various classes, but packages have limited support for visibility control. If you want classes and interfaces from one package to be visible to another package, you have to declare them as public. As a consequence, these classes and interfaces are accessible to everyone else as well. A typical occurrence of this problem is when you see companion packages with names that include the string `"impl"` to provide default implementations. In this case, because the code inside that package was defined as public, you have no way to prevent users from using these internal implementations. As a result, it becomes difficult to evolve your code without making breaking changes, because what you thought was for internal use only was used by a programmer temporarily to get something working and then frozen into the system. Worse, this situation is bad from a security point of view because you potentially increase the attack surface as more code is exposed to the risk of tampering.

ussed the benefits of software written in a way that makes it simple to maintain and understand—in other words, easier to reason about. We also talked about separation of concerns and modeling dependencies between modules. Unfortunately, Java historically falls short in supporting these ideas when it comes to bundling and

running an application. In fact, you have to ship all your compiled classes into one single flat JAR, which is made accessible from the class path.[1] Then the JVM can dynamically locate and load classes from the class path as needed.

1

This spelling is used in Java documentation, but `classpath` is often used for arguments to programs.

Unfortunately, the combination of the class path and JARs has several downsides.

First, the class path has no notion of versioning for the same class. You can't, for example, specify that the class JSONParser from a parsing library should belong to version 1.0 or version 2.0, so you can't predict what will happen if the same library with two different versions is available on the class path. This situation is common in large applications, as you may have different versions of the same libraries used by different components of your application.

Second, the class path doesn't support explicit dependencies; all the classes inside different JARs are merged into one bag of classes on the class path. In other words, the class path doesn't let you declare explicitly that one JAR depends on a set of classes contained inside another JAR. This situation makes it difficult to reason about the class path and to ask questions such as:

- Is anything missing?
- Are there any conflicts?

Build tools such as Maven and Gradle can help you solve this problem. Before Java 9, however, neither Java nor the JVM had any support for explicit dependencies. The issues combined are often referred to as JAR Hell or Class Path Hell. The direct consequence of these problems is that it's common to have to keep adding and removing class files on the class path in a trial-and-error cycle, in the hope that the JVM will execute your application without throwing runtime exceptions such as `ClassNotFoundException`. Ideally, you'd like such problems to be discovered early in g the Java 9 module system consistently encted at compile time.

Hell aren't problems only for your software architecture, however. What about the JDK itself?

14.2.2. Monolithic JDK

The *Java Development Kit* (JDK) is a collection of tools that lets you work with and run Java programs. Perhaps the most important tools you're familiar with are `javac` to compile Java programs and `java` to load and run a Java application, along with the JDK library, which provides runtime support including input/output, collections, and streams. The first version was released in 1996. It's important to understand that like any software, the JDK has grown and increased considerably in size. Many technologies were added and later deprecated. CORBA is a good example. It doesn't matter whether you're using CORBA in your application or not; its classes are shipped with the JDK. This situation becomes problematic especially in applications that run on mobile or in the cloud and typically don't need all the parts available in the JDK library.

How can you get away from this problem as a whole ecosystem? Java 8 introduced the notion of *compact profiles* as a step forward. Three profiles were introduced to have different memory footprints, depending on which parts of the JDK library you're interested in. Compact profiles, however, provided only a short-term fix. Many internal APIs in the JDK aren't meant for public use. Unfortunately, due to the poor encapsulation provided by the Java language, those APIs are commonly used. The class `sun.misc.Unsafe`, for example, is used by several libraries (including Spring, Netty, and Mockito) but was never intended to be made available outside the JDK internals. As a result, it's extremely difficult to evolve these APIs without introducing incompatible changes.

All these problems provided the motivation for designing a Java Module System that also can be used to modularize the JDK itself. In a nutshell, new structuring constructs were required to allow you to choose what parts of the JDK you need and how to reason about the class path, and to provide stronger encapsulation to evolve the platform.

14.2.3. Comparison with OSGi

This section compares Java 9 modules with OSGi. If you haven't heard of OSGi, we suggest that you skip this section.

Before the introduction of modules based on project Jigsaw into Java 9, Java already had a powerful module system, named OSGi, even if it wasn't formally part of the Java platform. The Open Service Gateway ini-
nd, until the arrival of Java 9, represented
ementing a modular application on the

In reality, OSGi and the new Java 9 Module System aren't mutually exclusive; they can coexist in the same application. In fact, their features over-

lap only partially. OSGi has a much wider scope and provides many capabilities that aren't available in Jigsaw.

OSGi modules are called *bundles* and run inside a specific OSGi framework. Several certified OSGi framework implementations exist, but the two with the widest adoption are Apache Felix and Equinox (which is also used to run the Eclipse IDE). When running inside an OSGi framework, a single bundle can be remotely installed, started, stopped, updated, and uninstalled without a reboot. In other words, OSGi defines a clear life cycle for bundles made by the states listed in **table 14.1**.

Table 14.1. Bundle states in OSGi

Bundle state	Description
INSTALLED	The bundle has been successfully installed.
RESOLVED	All Java classes that the bundle needs are available.
STARTING	The bundle is being started, and the BundleActivator.start method has been called, but the start method hasn't yet returned.
ACTIVE	The bundle has been successfully activated and is running.
STOPPING	The bundle is being stopped. The BundleActivator.stop method has been called, but the stop method hasn't yet returned.
UNINSTALLED	The bundle has been uninstalled. It can't move into another state.

The possibility of hot-swapping different subparts of your application without the need to restart it probably is the main advantage of OSGi ...fined through a text file describing which ...d by the bundle to work and which internal ...d by the bundle and then made available to other bundles.

Another interesting characteristic of OSGi is that it allows different versions of the same bundle to be installed in the framework at the same

time. The Java 9 Module System doesn't support version control because Jigsaw still uses one single class loader per application, whereas OSGi loads each bundle in its own class loader.

14.3. Java modules: the big picture

Java 9 provides a new unit of Java program structure: the *module*. A module is introduced with a new keyword[2] `module`, followed by its name and its body. Such a *module descriptor*[3] lives in a special file: `module-info.java`, which is compiled to `module-info.class`. The body of a module descriptor consists of clauses, of which the two most important are `requires` and `exports`. The former clause specifies what other modules your modules need to run, and `exports` specifies everything that your module wants to be visible for other modules to use. You learn about these clauses in more detail in later sections.

2

Technically, Java 9 module-forming identifiers—such as `module`, `requires`, and `export`—are restricted keywords. You can still use them as identifiers elsewhere in your program (for backward compatibility), but they're interpreted as keywords in a context where modules are allowed.

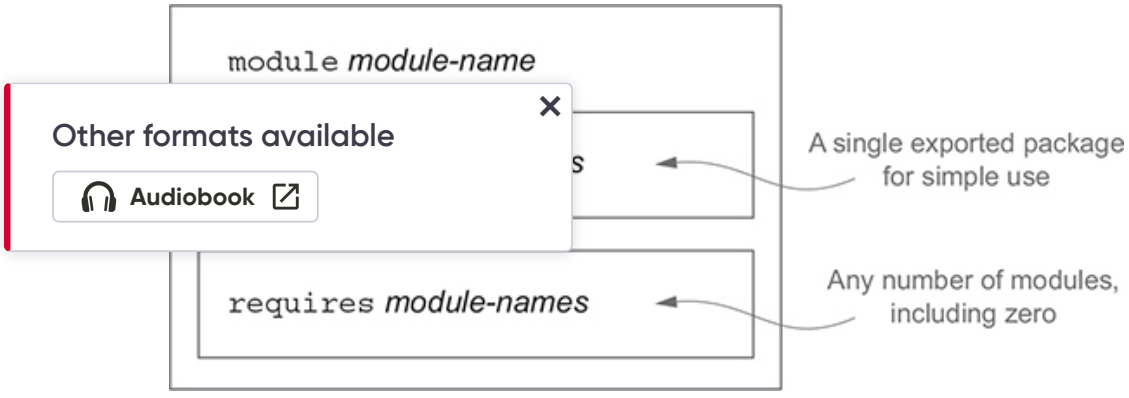
3

Legally, the textual form is called a module declaration, and the binary form in `module-info.class` is referred to as a module descriptor.

A module descriptor describes and encapsulates one or more packages (and typically lives in the same folder as these packages), but in simple use cases, it exports (makes visible) only one of these packages.

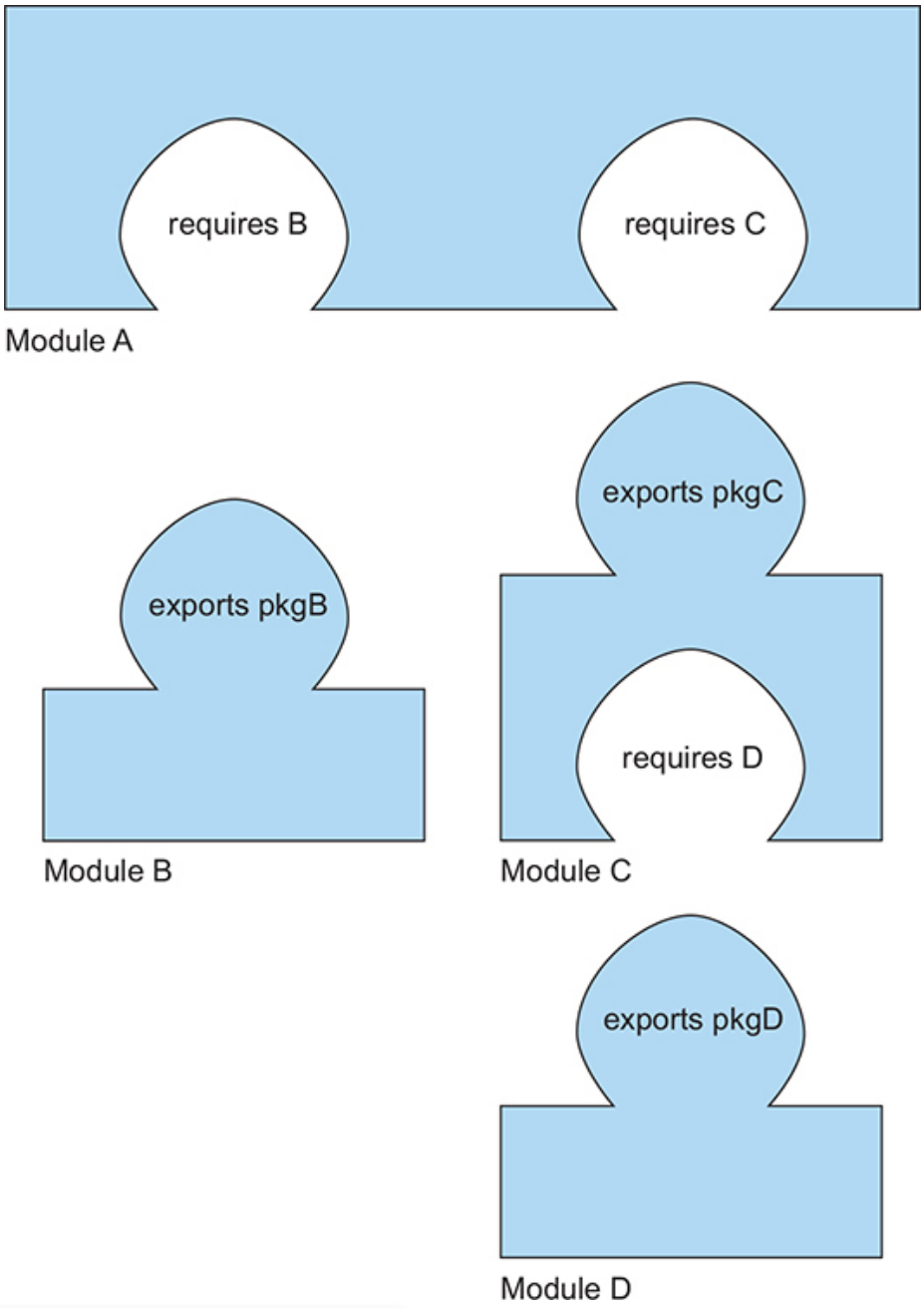
The core structure of a Java module descriptor is shown in **figure 14.2**.

Figure 14.2. Core structure of a Java module descriptor (`module-info.java`)



It is helpful to think of the `exports` and `requires` parts of a module as being respectively like the lugs (or tabs) and holes of a jigsaw puzzle (which is perhaps where the working name Project Jigsaw originated). Figure 14.3 shows an example with several modules.

Figure 14.3. Jigsaw-puzzle-style example of a Java system built from four modules (A, B, C, D). Module A requires modules B and C to be present, and thereby gets access to the packages `pkgB` and `pkgC` (exported by modules B and C, respectively). Module C may similarly use package `pkgD`, which it has required from module C, but Module B can't use `pkgD`.



…aven, much of the detail of module descrip-
…is hidden from the user.

Having said that, in the next section we explore these concepts in more detail based on examples.

14.4. Developing an application with the Java Module System

In this section, you get an overview of the Java 9 Module System by building a simple modular application from the ground up. You learn how to structure, package, and launch a small modular application. This section doesn't explain each topic in detail but shows you the big picture, so you can delve independently in more depth if needed.

14.4.1. Setting up an application

To get started with the Java Module System, you need an example project to write code for. Perhaps you're traveling a lot, grocery shopping, or going out for coffee with your friends, and you have to deal with a lot of receipts. Nobody ever enjoyed managing expenses. To help yourself out, you write an application that can manage your expenses. The application needs to conduct several tasks:

- Read a list of expenses from a file or a URL;
- Parse the string representations of these expenses;
- Calculate statistics;
- Display a useful summary;
- Provide a main start-up and close-down coordinator for these tasks.

You need to define different classes and interfaces to model the concepts in this application. First, a `Reader` interface lets you read serialized expenses from a source. You'll have different implementations, such as `HttpReader` or `FileReader`, depending on the source. You also need a `Parser` interface to deserialize the JSON objects into a domain object `Expense` that you can manipulate in your Java application. Finally, you need a class `SummaryCalculator` to be responsible for calculating statistics, given a list of `Expense` objects, and to return `SummaryStatistics` objects.

Now that you have a project, how do you modularize it by using the Java Module System? It's clear that the project involves several concerns, which you want to separate:

- Reading data from different sources (`Reader`, `HttpReader`, `FileReader`)
- Parsing the data from different formats (`Parser`, `JSONParser`, `ExpenseJSON–Parser`)
- ts (`Expense`)
- tatistics (`SummaryCalculator`,
- Coordinating the different concerns (`ExpensesApplication`)

Here, we'll take a fine-grained approach for pedagogic reasons. You can group each concern into a separate module, as follows (and we discuss

the module-naming scheme in more detail later):

- expenses.readers
- expenses.readers.http
- expenses.readers.file
- expenses.parsers
- expenses.parsers.json
- expenses.model
- expenses.statistics
- expenses.application

For this simple application, you adopt a fine-grained decomposition to exemplify the different parts of the module system. In practice, taking such a fine-grained approach for a simple project would result in a high upfront cost for the arguably limited benefit of properly encapsulating small parts of the project. As the project grows and more internal implementations are added, however, the encapsulation and reasoning benefits become more apparent. You could imagine the preceding list as being a list of packages, depending on your application boundaries. A module groups a series of packages. Perhaps each module has implementation-specific packages that you don't want to expose to other modules. The `expenses.statistics` module, for example, may contain several packages for different implementations of experimental statistical methods. Later, you can decide which of these packages to release to users.

14.4.2. Fine-grained and coarse-grained modularization

When you're modularizing a system, you can choose the granularity. In the most fine-grained scheme, every package has its own module (as in the previous section); in the most coarse-grained scheme, a single module contains all the packages in your system. As noted in the previous section, the first schema increases the design cost for limited gains, and the second one loses all benefits of modularization. The best choice is a pragmatic decomposition of the system into modules along with a regular review process to ensure that an evolving software project remains sufficiently modularized that you can continue to reason about it and modify it.

In short, modularization is the enemy of software rust.

asics

ar application, which has only one module
n. The project directory structure is as follows, with each level nested in a directory:

```
|— expenses.application
  |— module-info.java
```

```
|— com
  |— example
    |— expenses
      |— application
        |— ExpensesApplication.java
```

You've noticed this mysterious `module-info.java` that was part of the project structure. This file is a module descriptor, as we explained earlier in the chapter, and it must be located at the root of the module's source-code file hierarchy to let you specify the dependencies of your module and what you want to expose. For your expenses application, the top-level `module-info.java` file contains a module description that has a name but is otherwise empty because it neither depends on any other module nor exposes its functionality to other modules. You'll learn about more-sophisticated features later, starting with **section 14.5**. The content of `module-info.java` is as follows:

```
module expenses.application {

}
```

How do you run a modular application? Take a look at some commands to understand the low-level parts. This code is automated by your IDE and build system but seeing what's happening is useful. When you're in the module source directory of your project, run the following commands:

```
javac module-info.java
     com/example/expenses/application/ExpensesApplication.java -d targ

jar cvfe expenses-application.jar
     com.example.expenses.application.ExpensesApplication -C target
```

These commands produce output similar to the following, which shows which folders and class files are incorporated into the generated JAR (`expenses-application.jar`):

```
added manifest
                                    lule-info.class
                                    out= 0)(stored 0%)
                                    n = 0) (out= 0)(stored 0%)
                                    penses/(in = 0) (out= 0)(stored 0%)
adding: com/example/expenses/application/(in = 0) (out= 0)(stored 0%)
adding: com/example/expenses/application/ExpensesApplication.class(in
     (out= 306)(deflated 32%)
```

Finally, you run the generated JAR as a modular application:

```
java --module-path expenses-application.jar \
     --module expenses/com.example.expenses.application.ExpensesApplic
```

You should be familiar with the first two steps, which represent a standard way to package a Java application into a JAR. The only new part is that the file `module-info.java` becomes part of the compilation step.

The `java` program, which runs Java `.class` files, has two new options:

- `--module-path`—This option specifies what modules are available to load. This option differs from the `--classpath` argument, which makes class files available.
- `--module`—This option specifies the main module and class to run.

A module's declaration doesn't include a version string. Addressing the version-selection problem wasn't a specific design point for the Java 9 Module System, so versioning isn't supported. The justification was that this problem is one for build tools and container applications to address.

14.5. Working with several modules

Now that you know how to set up a basic application with one module, you're ready to do something a bit more realistic with multiple modules. You want your expense application to read expenses from a source. To this end, introduce a new module `expenses.readers` that encapsulates these responsibilities. The interaction between the two modules `expenses.application` and `expenses.readers` is specified by the Java 9 `exports` and `requires` clauses.

14.5.1. The exports clause

Here's how we might declare the module `expenses.readers`. (Don't worry about the syntax and concepts yet; we cover these topics later.)

```
module expenses.readers {

                    le.expenses.readers;        1
                    e.expenses.readers.file;    1
                    e.expenses.readers.http;    1
```


- ***1*** **These are package names, not module names.**

There's one new thing: the `exports` clause, which makes the public types in specific packages available for use by other modules. By default, everything is encapsulated within a module. The module system takes a whitelist approach that helps you get strong encapsulation, as you need to explicitly decide what to make available for another module to use. (This approach prevents you from accidentally exporting some internal features that a hacker can exploit to compromise your systems several years later.)

The directory structure of the two-module version of your project now looks like this:

```
|— expenses.application
  |— module-info.java
  |— com
    |— example
      |— expenses
        |— application
          |— ExpensesApplication.java

|— expenses.readers
  |— module-info.java
  |— com
    |— example
      |— expenses
        |— readers
          |— Reader.java
        |— file
          |— FileReader.java
        |— http
          |— HttpReader.java
```

14.5.2. The requires clause

Alternatively, you could have written `module-info.java` as follows:

```
module expenses.readers {
    requires java.base;                          1

    exports com.example.expenses.readers;        2
    exports com.example.expenses.readers.file;   2
    exports com.example.expenses.readers.http;   2
}
```


- *1* **This is a module name, not a package name.**
- *2* **This is a package name, not a module name.**

The new element is the `requires` clause, which lets you specify what the module depends on. By default, all modules depend on a platform module called `java.base` that includes the Java main packages such as `net`, `io`, and `util`. This module is always required by default, so you don't need to say so explicitly. (This is similar to how saying `"class Foo { ... }"` in Java is equivalent to saying `"class Foo extends Object { ... }"`).

It becomes useful when you need to import modules other than `java.base`.

The combination of the `requires` and `exports` clauses makes access control of classes more sophisticated in Java 9. **Table 14.2** summarizes the differences in visibility with the different access modifiers before and after Java 9.

Table 14.2. Java 9 provides finer control over class visibility

Class visibility	Before Java 9	After Java 9
All classes public to everyone	✓✓	✓✓ (combination of exports and requires clauses)
Limited number of classes public	✗✗	✓✓ (combination of exports and requires clauses)
Public inside one module only	✗✗	✓ (no exports clause)
Protected	✓✓	✓✓
	✓	✓✓
Private	✓✓	✓✓

14.5.3. Naming

At this stage, it's useful to comment on the naming convention for modules. We went for a short approach (for example, `expenses.application`) so as not to confuse the ideas of modules and packages. (A module can export multiple packages.) The recommended convention is different, however.

Oracle recommends you name modules following the same reverse internet domain-name convention (for example, com.iteratrlearning.training) used for packages. Further, a module's name should correspond to its principal exported API package, which should also follow that convention. If a module doesn't have that package, or if for other reasons it requires a name that doesn't correspond to one of its exported packages, it should start with the reversed form of an internet domain associated with its author.

Now that you've learned how to set up a project with multiple modules, how do you package and run it? We cover this topic in the next section.

14.6. Compiling and packaging

Now that you're comfortable with setting a project and declaring a module, you're ready to see how you can use build tools like Maven to compile your project. This section assumes that you're familiar with Maven, which is one of the most common build tools in the Java ecosystem. Another popular building tool is Gradle, which we encourage you to explore if you haven't heard of it.

First, you need to introduce a `pom.xml` file for each module. In fact, each module can be compiled independently so that it behaves as a project on its own. You also need to add a `pom.xml` for the parent of all the modules to coordinate the build for the whole project. The overall structure now looks as follows:

```
|— pom.xml
|— expenses.application
  |— pom.xml
  |— src
    |— main
      |— java
                                    .java
                                    s
                  |— application
                    |— ExpensesApplication.java
|— expenses.readers
  |— pom.xml
  |— src
```

```
|— main
  |— java
    |— module-info.java
    |— com
      |— example
        |— expenses
          |— readers
            |— Reader.java
          |— file
            |— FileReader.java
          |— http
            |— HttpReader.java
```

Notice the three new `pom.xml` files and the Maven directory project structure. The module descriptor (`module-info.java`) needs to be located in the `src/main/java` directory. Maven will set up `javac` to use the appropriate module source path.

The `pom.xml` for the `expenses.readers` project looks like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
     http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>expenses.readers</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>
    <parent>
        <groupId>com.example</groupId>
        <artifactId>expenses</artifactId>
        <version>1.0</version>
    </parent>
</project>
```

The important thing to note is that this code explicitly mentions the parent module to help in the build process. The parent is the artifact with the ID `expenses`. You need to define the parent in `pom.xml`, as you see

`om.xml` for the `expenses.application` module. This file is similar to the preceding one, but you have to add a dependency to the `expenses.readers` project, because `ExpensesApplication` requires the classes and interfaces that it contains to compile:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
     http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>expenses.application</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>

    <parent>
        <groupId>com.example</groupId>
        <artifactId>expenses</artifactId>
        <version>1.0</version>
    </parent>

    <dependencies>
        <dependency>
                <groupId>com.example</groupId>
                <artifactId>expenses.readers</artifactId>
                <version>1.0</version>
        </dependency>
    </dependencies>

</project>
```

Now that two modules, `expenses.application` and `expenses.readers`, have their own `pom.xml`, you can set up the global `pom.xml` to guide the build process. Maven supports projects that have multiple Maven modules with the special XML element `<module>`, which refers to the children's artifact IDs. Here's the complete definition, which refers to the two child modules `expenses.application` and `expenses.readers`:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
     http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>expenses</artifactId>
    <packaging>pom</packaging>
    <version>1.0</version>

    <modules>
    <module>expenses.application</module>
        <module>expenses.readers</module>
```

```
    </modules>

    <build>
        <pluginManagement>
            <plugins>
            <plugin>
                    <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                    <version>3.7.0</version>
                <configuration>
                        <source>9</source>
                        <target>9</target>
                </configuration>
            </plugin>
            </plugins>
        </pluginManagement>
    </build>
</project>
```

Congratulations! Now you can run the command `mvn clean package` to generate the JARs for the modules in your project. This command generates

```
./expenses.application/target/expenses.application-1.0.jar
./expenses.readers/target/expenses.readers-1.0.jar
```

You can run your module application by including these two JARs on the module path as follows:

```
java --module-path \
 ./expenses.application/target/expenses.application-1.0.jar:\
 ./expenses.readers/target/expenses.readers-1.0.jar \
    --module \
 expenses.application/com.example.expenses.application.ExpensesApplica
```

So far, you've learned about modules you created, and you've seen how to use `requires` to reference `java.base`. Real-world software, however, depends on external modules and libraries. How does that process work, and what if legacy libraries haven't been updated with an explicit `mod-`

You may decide that the implementation of your `HttpReader` is low-level; instead, you'd like to use a specialized library such as the `httpclient` from the Apache project. How do you incorporate that library into your

project? You've learned about the `requires` clause, so try to add it in the `module-info.java` for the `expenses.readers` project. Run `mvn clean package` again to see what happens. Unfortunately, the result is bad news:

```
[ERROR] module not found: httpclient
```

You get this error because you also need to update your `pom.xml` to state the dependency. The maven compiler plugin puts all dependencies on the module path when you're building a project that has a `module-info.java` so that the appropriate JARs are downloaded and recognized in your project, as follows:

```
<dependencies>
    <dependency>
        <groupId>org.apache.httpcomponents</groupId>
        <artifactId>httpclient</artifactId>
        <version>4.5.3</version>
    </dependency>
</dependencies>
```

Now running `mvn clean package` builds the project correctly. Notice something interesting, though: the library `httpclient` isn't a Java module. It's an external library that you want to use as a module, but it hasn't yet been modularized. Java turns the appropriate JAR into a so-called automatic module. Any JAR on the module path without a `module-info` file becomes an automatic module. Automatic modules implicitly export all their packages. A name for this automatic module is invented automatically, derived from the JAR name. You have a few ways to derive the name, but the easiest way is to use the `jar` tool with the `--describe-module` argument:

```
jar --file=./expenses.readers/target/dependency/httpclient-4.5.3.jar \
    --describe-module
httpclient@4.5.3 automatic
```

In this case, the name is `httpclient`.

pplication and adding the `httpclient` JAR

```
java --module-path \
 ./expenses.application/target/expenses.application-1.0.jar:\
 ./expenses.readers/target/expenses.readers-1.0.jar \
```

```
./expenses.readers/target/dependency/httpclient-4.5.3.jar \
  --module \
expenses.application/com.example.expenses.application.ExpensesApplica
```

Note

There's a project (**https://github.com/moditect/moditect**) to provide better support for the Java 9 Module System within Maven, such as to generate `module-info` files automatically.

14.8. Module declaration and clauses

The Java Module System is a large beast. As we mentioned earlier, we recommend that you read a dedicated book on the topic if you'd like to go further. Nonetheless, this section gives you a brief overview of other keywords available in the module declaration language to give you an idea of what's possible.

As you learned in the earlier sections, you declare a module by using the `module` directive. Here, it has the name `com.iteratrlearning.application`:

```
module com.iteratrlearning.application {

}
```

What can go inside the module declaration? You've learned about the `requires` and `exports` clauses, but there are other clauses, including `requires-transitive`, `exports-to`, `open`, `opens`, `uses`, and `provides`. We look at these clauses in turn in the following sections.

14.8.1. requires

The `requires` clause lets you specify that your module depends on another module at both compile time and runtime. The module `com.itera-` ...r example, depends on the module


```
module com.iteratrlearning.application {
    requires com.iteratrlearning.ui;
}
```

The result is that only public types that were exported by `com.iteratrlearning.ui` are available for `com.iteratrlearning.application` to use.

14.8.2. exports

The `exports` clause makes the public types in specific packages available for use by other modules. By default, no package is exported. You gain strong encapsulation by making explicit what packages should be exported. In the following example, the packages `com.iteratrlearning.ui.panels` and `com.iteratrlearning.ui.widgets` are exported. (Note that `exports` takes a *package name* as an argument and that `requires` takes a *module name*, despite the similar naming schemes.)

```
module com.iteratrlearning.ui {
    requires com.iteratrlearning.core;
    exports com.iteratrlearning.ui.panels;
    exports com.iteratrlearning.ui.widgets;
}
```

14.8.3. requires transitive

You can specify that a module can use the public types required by another module. You can modify the `requires` clause, for example, to `requires-transitive` inside the declaration of the module `com.iteratrlearning.ui`:

```
module com.iteratrlearning.ui {
    requires transitive com.iteratrlearning.core;

    exports com.iteratrlearning.ui.panels;
    exports com.iteratrlearning.ui.widgets;
}

module com.iteratrlearning.application {
    requires com.iteratrlearning.ui;
}
```

The result is that the module `com.iteratrlearning.application` has
rted by `com.iteratrlearning.core`.
module required (here, `com.iteratr-`
om another module required by this module (`com.iteratrlearning.core`). It would be annoying to re-declare `requires com.iteratrlearning.core` inside the module `com.iteratrlearning.application`. This problem is solved by `transitive`. Now

any module that depends on `com.iteratrlearning.ui` automatically reads the `com.iteratrlearning.core` module.

14.8.4. exports to

You have a further level of visibility control, in that you can restrict the allowed users of a particular export by using the `exports to` construct. As you saw in **section 14.8.2**, you can restrict the allowed users of `com.iteratrlearning.ui.widgets` to `com .iteratrlearning.ui.widgetuser` by adjusting the module declaration like so:

```
module com.iteratrlearning.ui {
    requires com.iteratrlearning.core;

    exports com.iteratrlearning.ui.panels;
    exports com.iteratrlearning.ui.widgets to
     com.iteratrlearning.ui.widgetuser;
}
```

14.8.5. open and opens

Using the `open` qualifier on module declaration gives other modules reflective access to all its packages. The `open` qualifier has no effect on module visibility except for allowing reflective access, as in this example:

```
open module com.iteratrlearning.ui {

}
```

Before Java 9, you could inspect the private state of objects by using reflection. In other words, nothing was truly encapsulated. Object-relational mapping (ORM) tools such as Hibernate often use this capability to access and modify state directly. In Java 9, reflection is no longer allowed by default. The `open` clause in the preceding code serves to allow that behavior when it's needed.

Instead of opening a whole module to reflection, you can use an `opens` clause within a module declaration to open its packages individually, as
to qualifier in the `opens-to` variant to
erform reflective access, analogous to how
s allowed to `require` an exported package.

14.8.6. uses and provides

If you're familiar with services and `ServiceLoader`, the Java Module System allows you to specify a module as a service provider using the

`provides` clause and a service consumer using the `uses` clause. This topic is advanced, however, and beyond the scope of this chapter. If you're interested in combining modules and service loaders, we recommend that you read a comprehensive resource such as *The Java Module System*, by Nicolai Parlog (Manning Publications), mentioned earlier in this chapter.

14.9. A bigger example and where to learn more

You can get a flavor of the module system from the following example, taken from Oracle's Java documentation. This example shows a module declaration using most of the features discussed in this chapter. The example isn't meant to frighten you (the vast majority of module statements are simple exports and requires), but it gives you a look at some richer features:

```
module com.example.foo {
    requires com.example.foo.http;
    requires java.logging;

    requires transitive com.example.foo.network;

    exports com.example.foo.bar;
    exports com.example.foo.internal to com.example.foo.probe;

    opens com.example.foo.quux;
    opens com.example.foo.internal to com.example.foo.network,
                                      com.example.foo.probe;

    uses com.example.foo.spi.Intf;
    provides com.example.foo.spi.Intf with com.example.foo.Impl;
}
```

This chapter discussed the need for the new Java Module System and provided a gentle introduction to its main features. We didn't cover many features, including service loaders, additional module descriptor clauses, and tools for working with modules such as `jdeps` and `jlink`. If you're a Java EE developer, it's important to keep in mind when migrating your applications to Java 9 that several packages relevant to EE aren't loaded by default in the modularized Java 9 Virtual Machine. The JAXB API classdered to be Java EE APIs and are no longer ath in Java SE 9. You need to explicitly add e `--add-modules` command-line switch to keep compatibility. To add `java.xml.bind`, for example, you need to specify `--add-modules java.xml.bind`.

As we noted earlier, doing the Java Module System justice would require a whole book, not a single chapter. To explore the details in greater depth, we suggest a book such as *The Java Module System*, by Nicolai Parlog (Manning Publications), mentioned earlier in this chapter.

Summary

- Separation of concerns and information hiding are two important principles to help construct software that you can reason about.
- Before Java 9, you made code modular by introducing packages, classes, and interfaces that have a specific concern, but these elements weren't rich enough for effective encapsulation.
- The Class Path Hell problem makes it hard to reason about the dependencies of an application.
- Before Java 9, the JDK was monolithic, resulting in high maintenance costs and restricted evolution.
- Java 9 introduced a new module system in which a `module-info.java` file names a module and specifies its dependencies (`requires`) and public API (`exports`).
- The `requires` clause lets you specify dependencies on other modules.
- The `exports` clause makes the public types of specific packages in a module available for use by other modules.
- The prefered naming convention for a module follows the reverse internet domain-name convention.
- Any JAR on the module path without a `module-info` file becomes an automatic module.
- Automatic modules implicitly export all their packages.
- Maven supports applications structured with the Java 9 Module System.